CS & IT ENGINEERING



Algorithms

Graph Algorithms

Lecture No.- 02



By- Aditya sir

Recap of Previous Lecture



Topic Graph Traversals ✓

Topic DFS/DFT → discovery & finishing

BFS ✓

Topics to be Covered



Topic DFS

Topic
1. Directed graph
   DAG
   Topological Sort

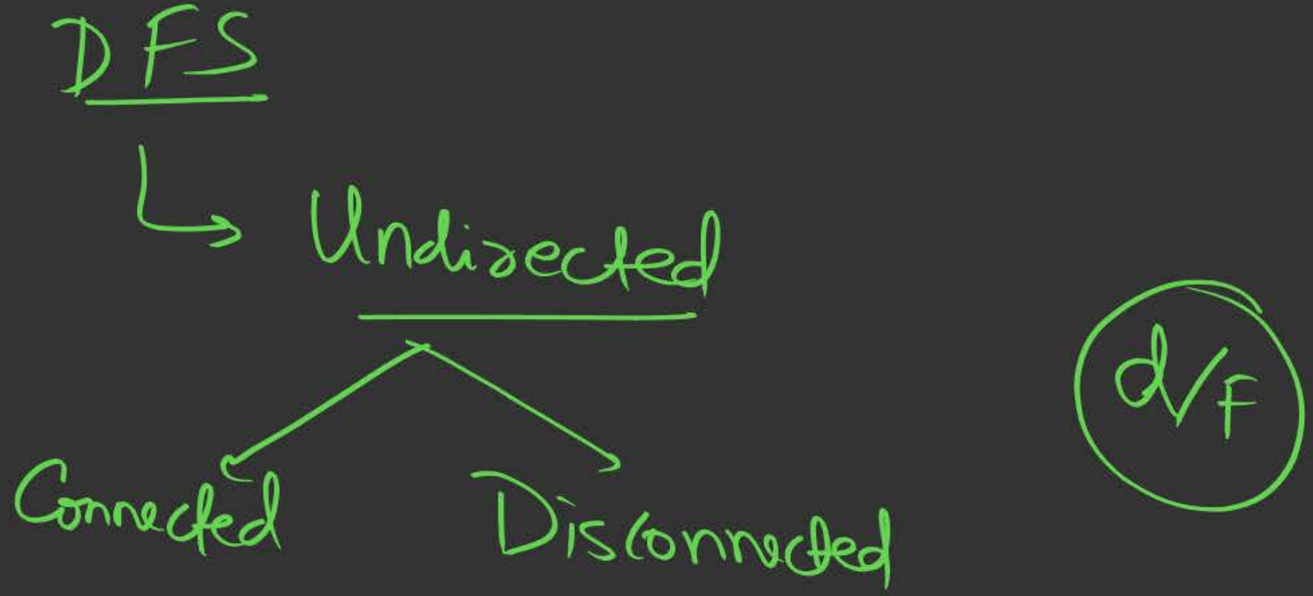
ABOUT ME

Hello, I'm Aditya.

1. Represented college as the first Google DSC Ambassador.
2. The only student from the batch to secure an internship at Amazon.
3. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis8. Completed my Masters with an overall GPA of 9.36/108. Joined Dream11 as a Data Scientist.9. Have mentored around 10k working professions in field of Data Science and Analytics10. Have been mentoring GATE aspirants to secure a great rank in limited time.11. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



By- Aditya sir


DFS
Undirected
Connected
Disconnected
dVf



Topic : Graph Algorithms

DFS in directed graphs:

Terminologies:-

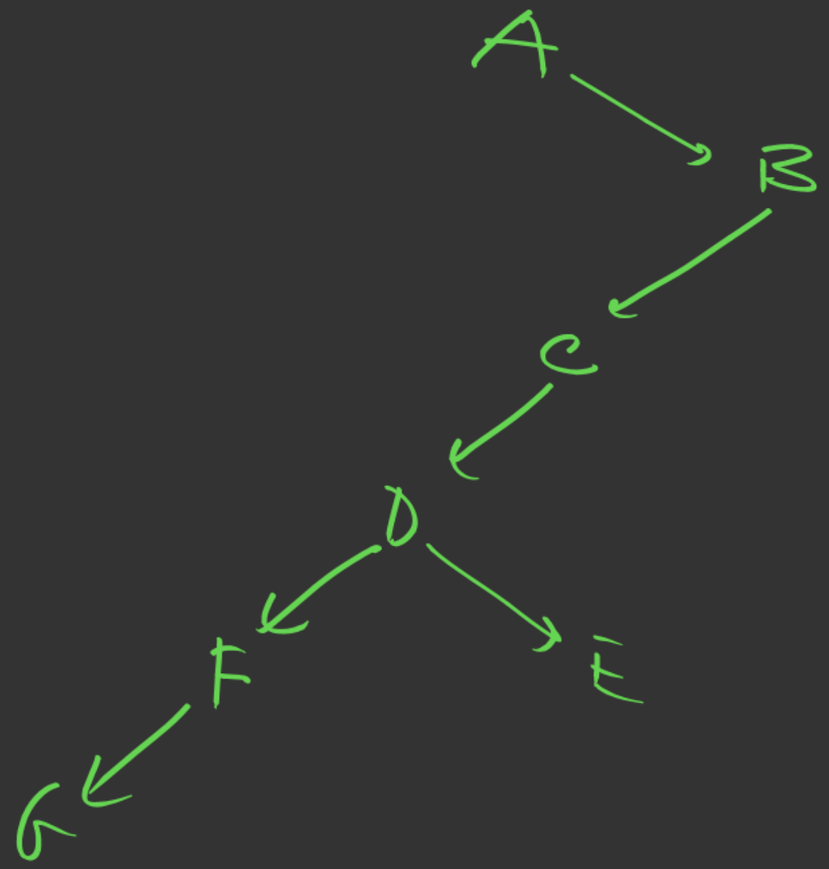
Whenever we perform DFS on a directed graph, it can lead to the different types of edges listed before.

1. **Tree Edge:**

Tree edge are those edges that are present in the DFS spanning tree tree/forest:

The edges that are not present in DFS spanning tree/forest can be 3 types:

1. Forward
2. Backward
3. Cross


A
B
C
D
F
E
G



Topic : Graph Algorithms

2. Forward edge:-

It is the edge that is going form a node to its non-child ~~discontents.~~ descendents

3. Back/Backward edge:

This is an edge from a node to its ~~acnes for:~~ ancestor

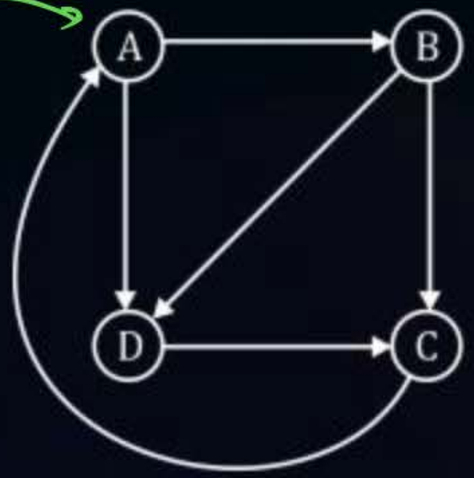
4. Cross edge:-

This is the edge from one node to some other node that is neither its ~~acnes for~~ ancestor

not its ~~discontent~~ descendent

Topic : Graph Algorithms

E.g.1. Given G(V,E)



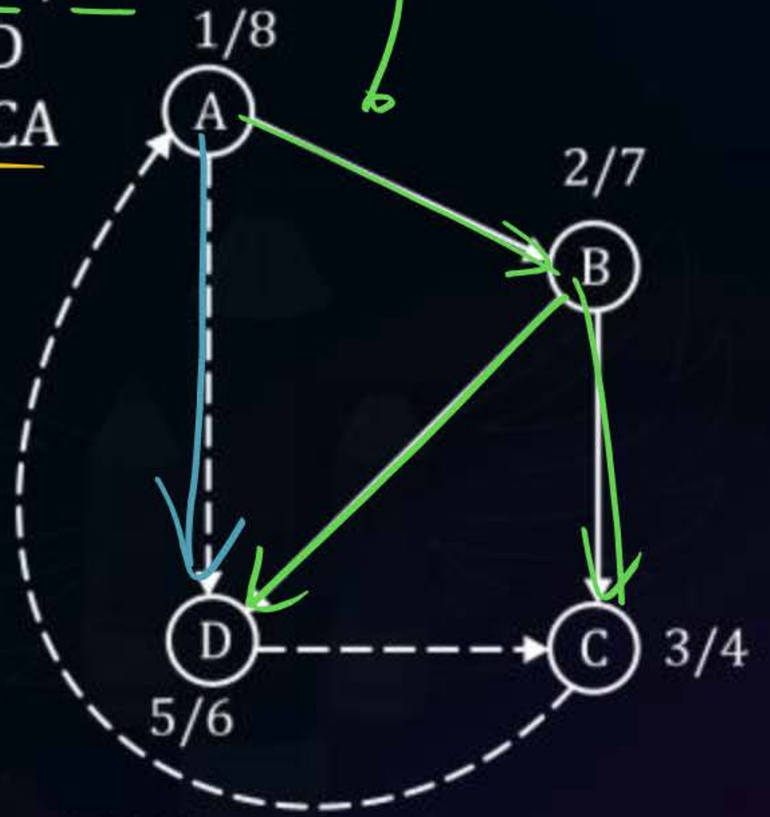
DFS

Apply DFS starting at A-

1. DFS Sorting

Types of edges

1. Tree edges:-AB, BC, BD
2. Forward edge:-AD
3. Backward edge:-CA
4. Cross edge:-DC



DFS Spanning Tree

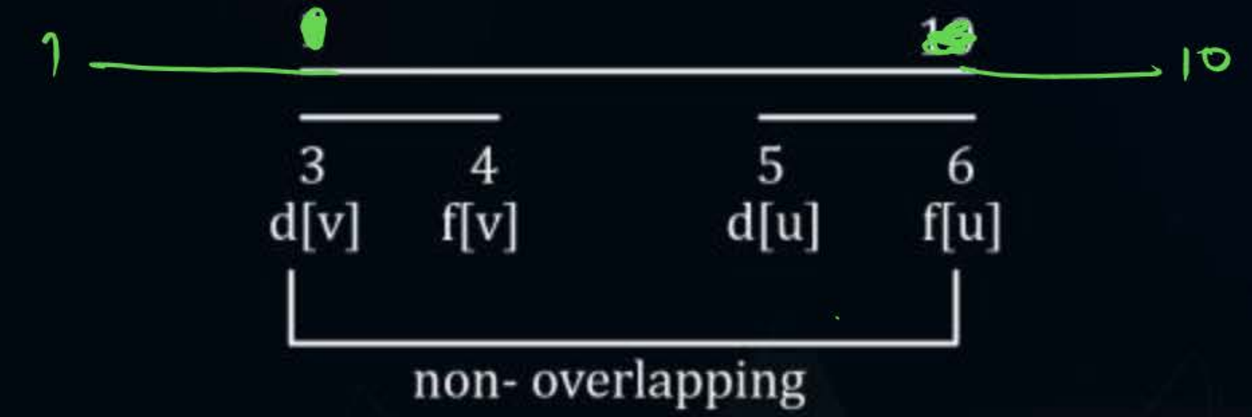


Topic : Graph Algorithms

E.g.2. 1. D→ C

D: d: 5/6

C: V : 3/4

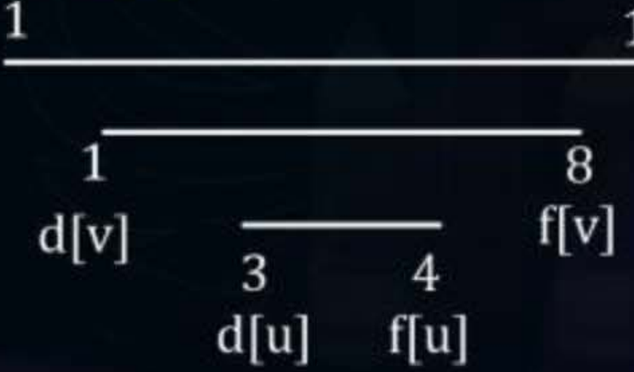


2. u → v

C → A

C: d[u] /f[u]: 3/4

C: d[v] /f[v]: 1/8



u → v back edge



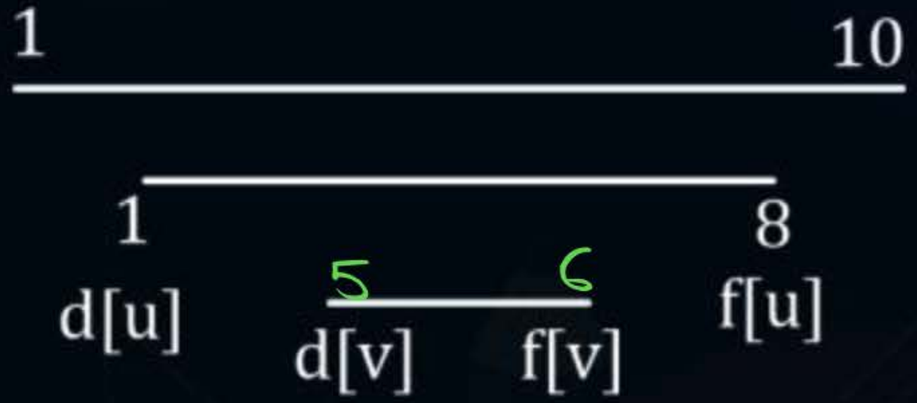
Topic : Graph Algorithms

E.g.3. $u \rightarrow v$ (A → D)

u: A: d[u]/ f[u]: 1/8

v: D: d[v]/ f[v]: 5/6

u → v : forward edge.





Topic : Graph Algorithms

Note. For same graph for same starting vertex different DFS spanning Trees are possible and hence, the types of edges can also be different.

Obs: Tree edge
+
Forward edge
+
Backward edges
+
Cross edge

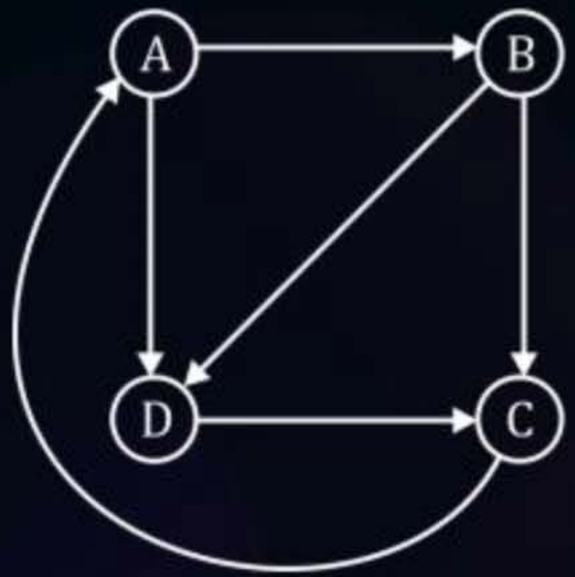
} Graph



Topic : Graph Algorithms

E.g. 4.

Given G (V,E)

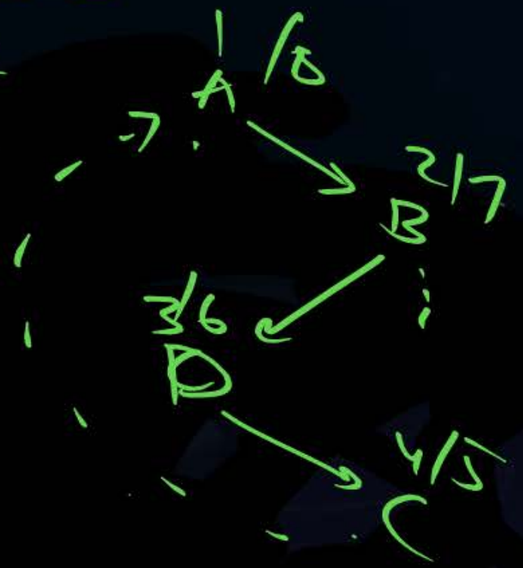


Apply DFS starting at A.

2. DFS Sorting at A

Types of edges

1. Tree edges:- AB, BD, DC
2. Forward edge:- AD, BC
3. Backward edge. CA
4. Cross edge:- None





Topic : Graph Algorithms

Imp

d(u) → discovery
F(u) → Finishing

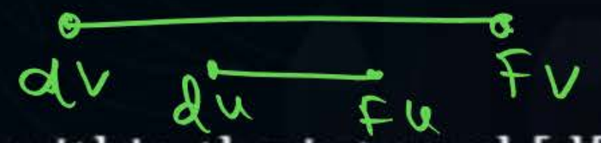
In any Depth-First Search of a (directed or undirected) graph G = (V, E), for any two vertices u and v exactly one of the following three conditions holds:

I. The intervals [d[u], f[u]] and [d[v], f[v]] are entirely disjoint, and neither u nor v is a descendant of the other in the depth-first forest,

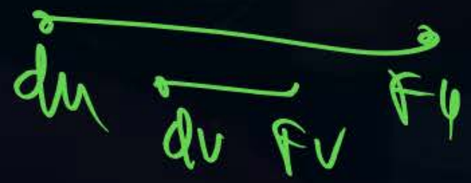
non-overlap

u → v : Cross edge

II. The interval [d[u], f[u]] is contained entirely within the interval [d[v], f[v]], and u is a descendant of v in a depth-first tree, or



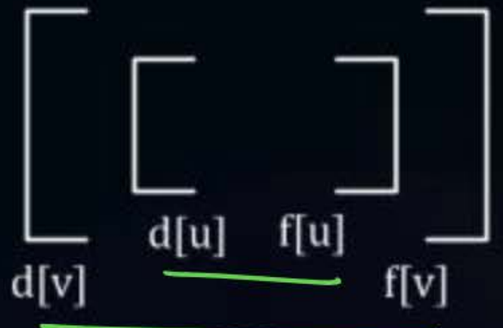
III. The interval [d[v], f[v]] is contained entirely within the interval [d[u], f[u]], and v is a descendant of u in a depth-first tree.

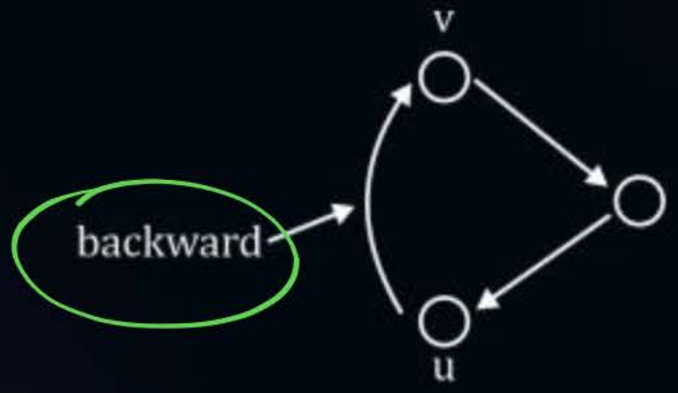




Topic : Graph Algorithms

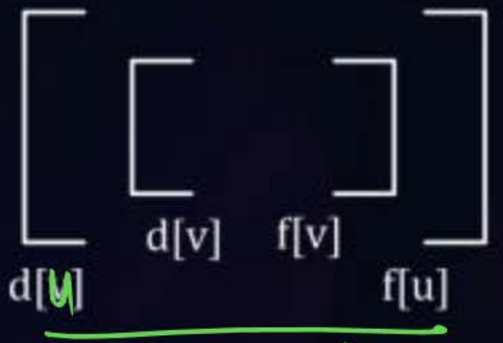
2. Backward u→ v:

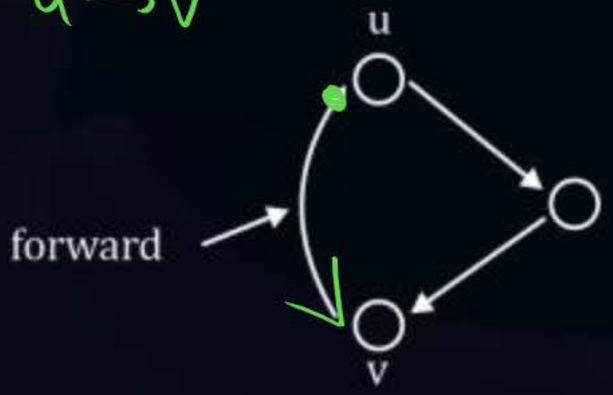




3. Forward u→ v

u → v







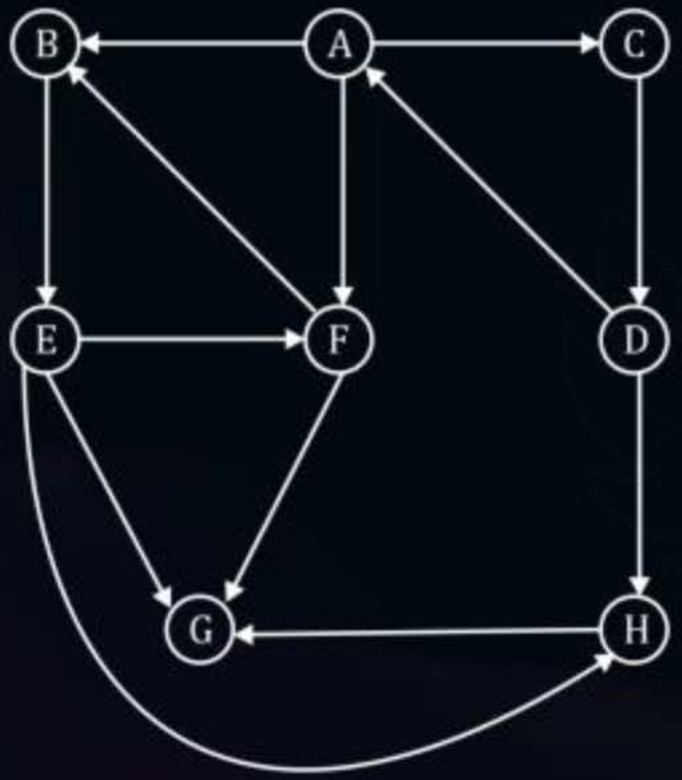
Topic : Graph Algorithms

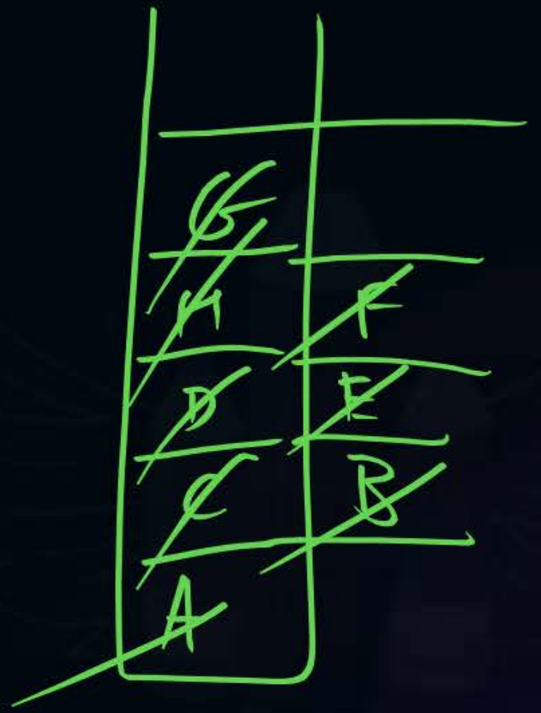
Test: End to end practice:

Give: G (V,E)

Start at A

13







Topic : Graph Algorithms

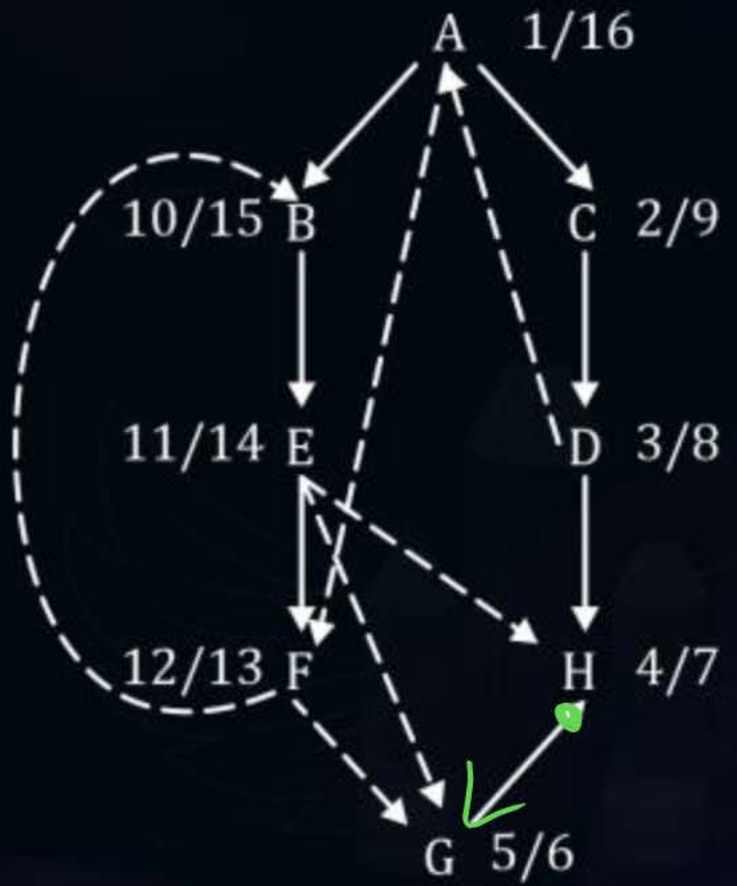
DFS starting at A

Type of edges:

1. Tree edges: AC, CD, DH, HG AB, BE, EF.
2. Froward edges: AF
3. Backward edges: DA, FB
4. Cross edge: FG, [illegible] EH

13

, EG,



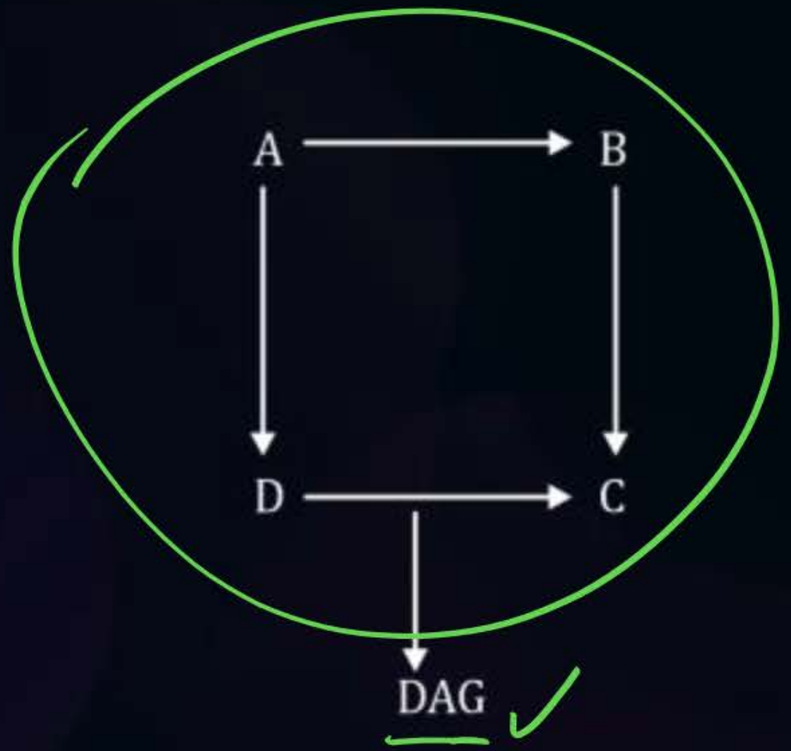

DAG

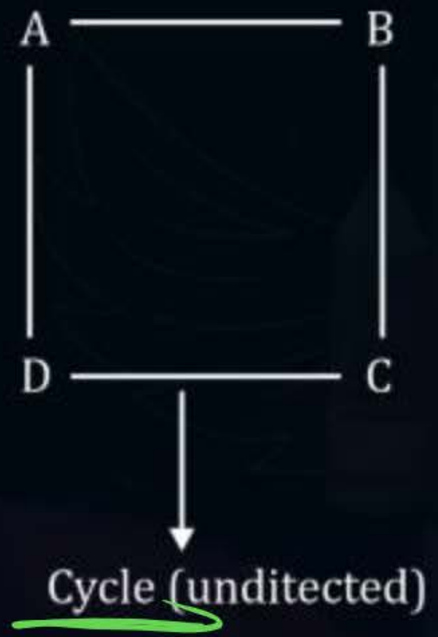


Topic : Graph Algorithms

DFS in directed acyclic graph: (DAG)

DAG: Directed graph without any cycle in it.







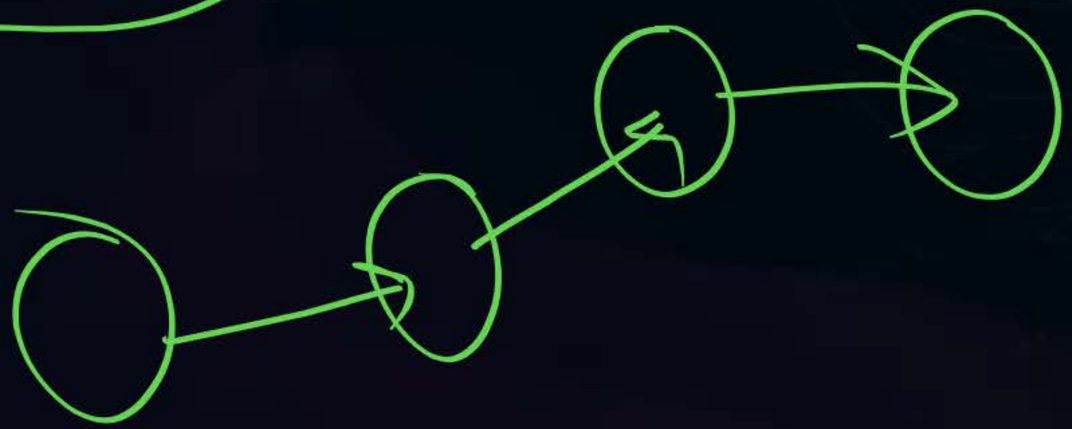
Topic : Graph Algorithms

Topological Sorts: Application of DFS on a directed a cycle graph DAG

Ordering

Definition:

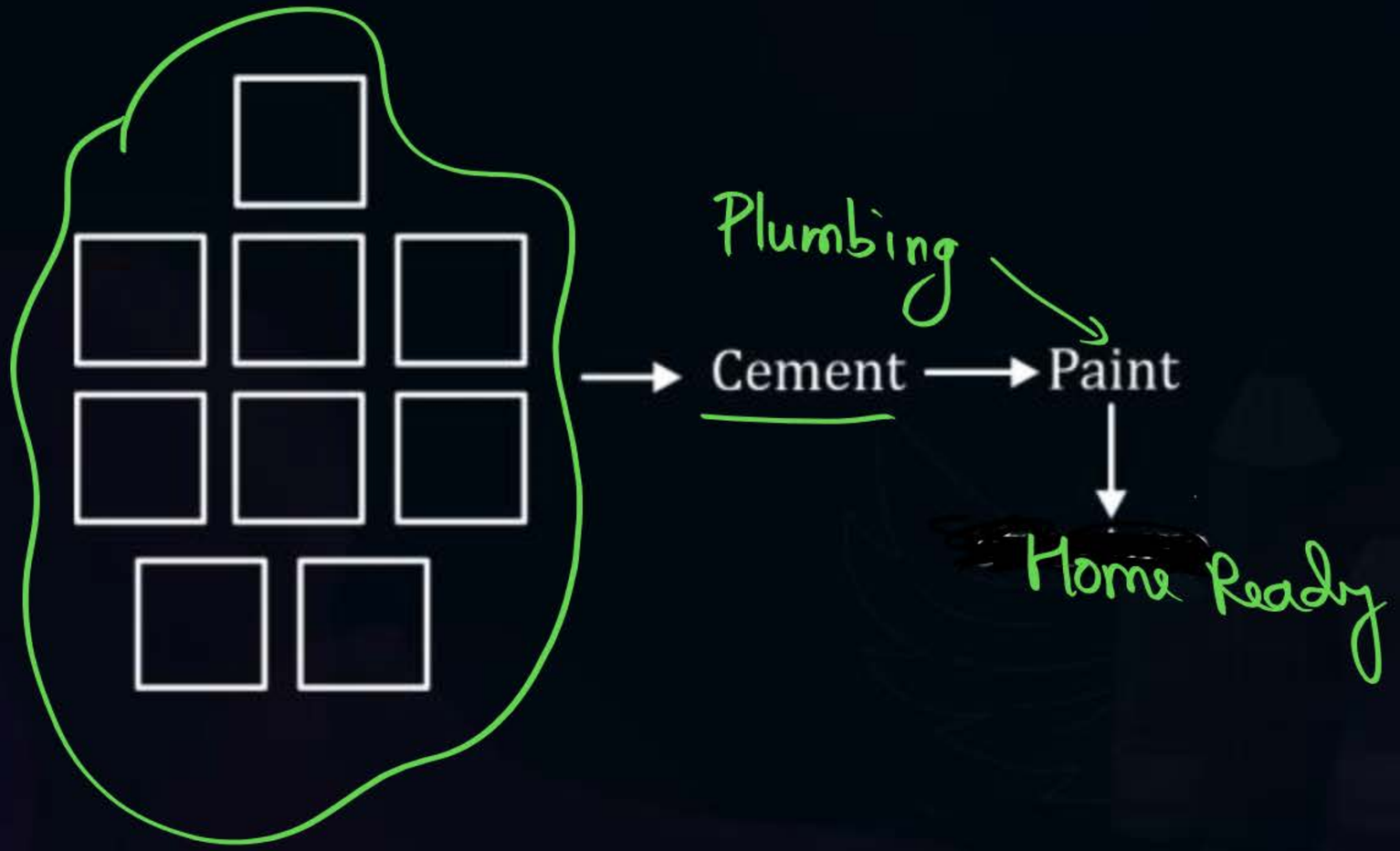
It is the linear order of vertices/nodes that represent some activities that has some precedencies.





Topic : Graph Algorithms

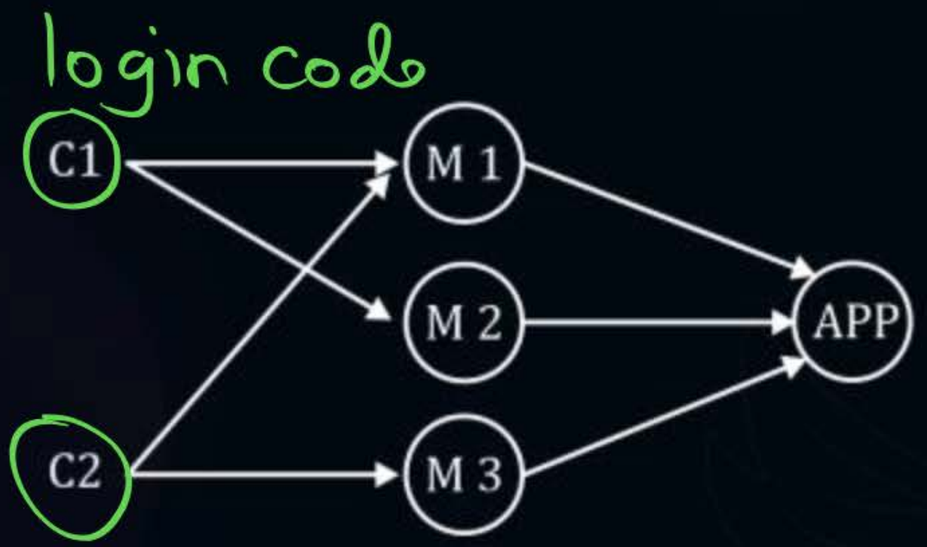
E.g.5.





Topic : Graph Algorithms

E.g.6.

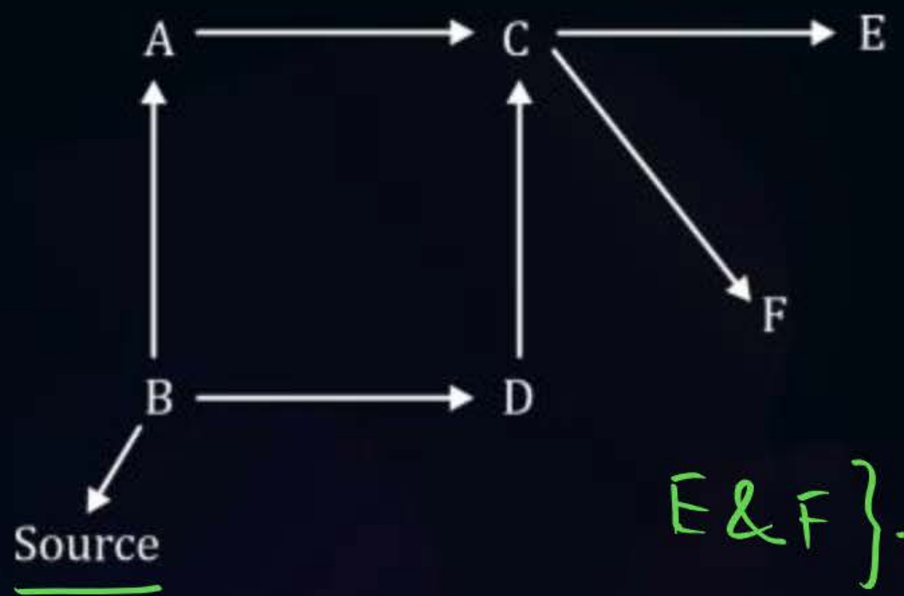




Topic : Graph Algorithms

E.g.7. Given a DAG

DAG



Sink: Outdegree (no. of outgoing edge =0)

Source: indegree (no. of incoming edges =0)

Note:- We visit a node only when all of its precedencies nodes are visited

E.g.8. D is precedence of C.

E & F } → Sink

Topological sort

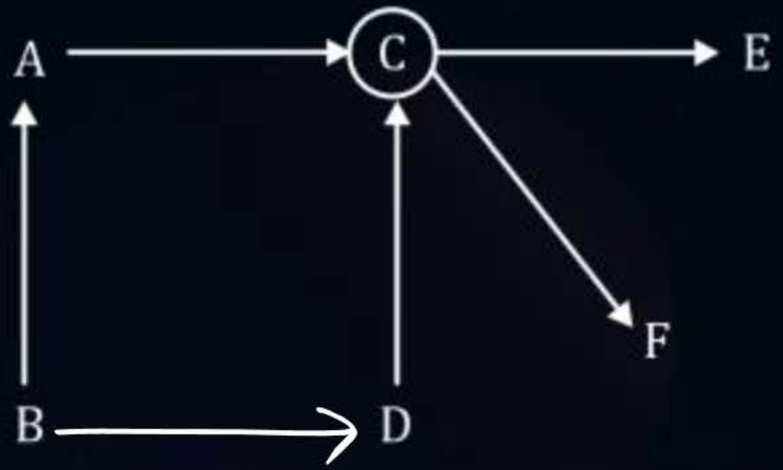
Source→ Sink



Topic : Graph Algorithms

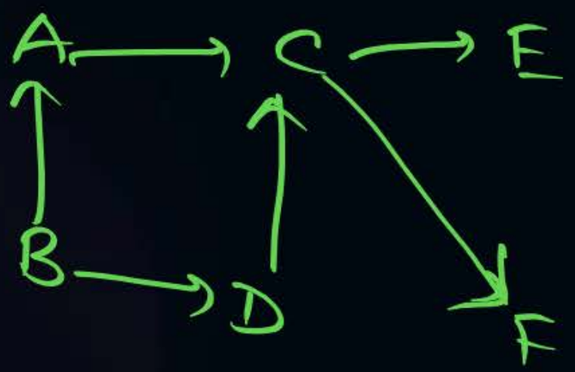
Finding all possible Topological orderings

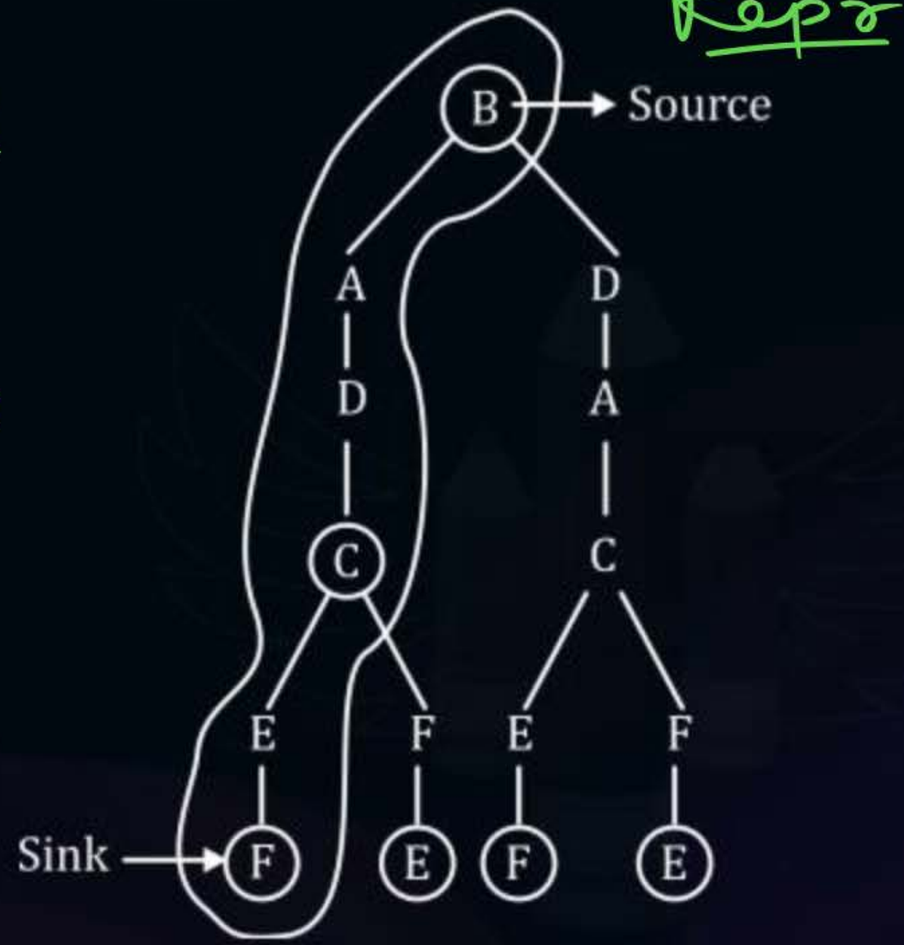
Given DAG



Topic : Graph Algorithms

#Q. Every branch from source → Sink will give I possible topological order.





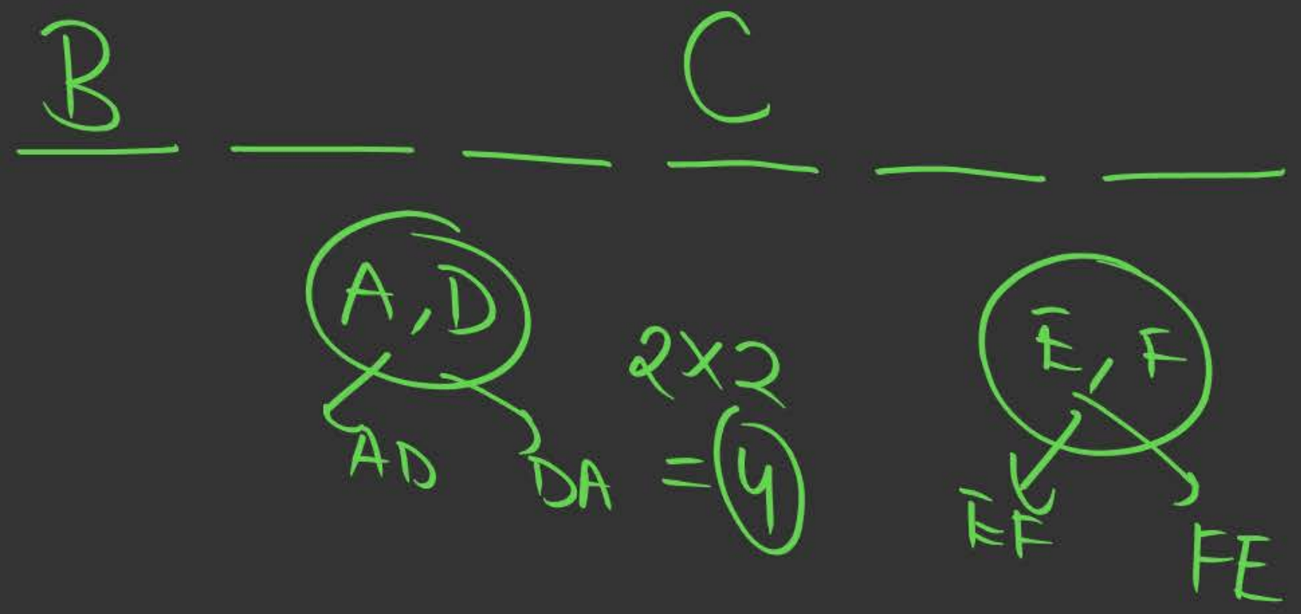
A BADCEF

B BADCFE

C BDACEF

D BDACFE

Shortcut





Topic : Graph Algorithms

Algorithm for Topological sort using DFS:

Algo Topological sort (G)

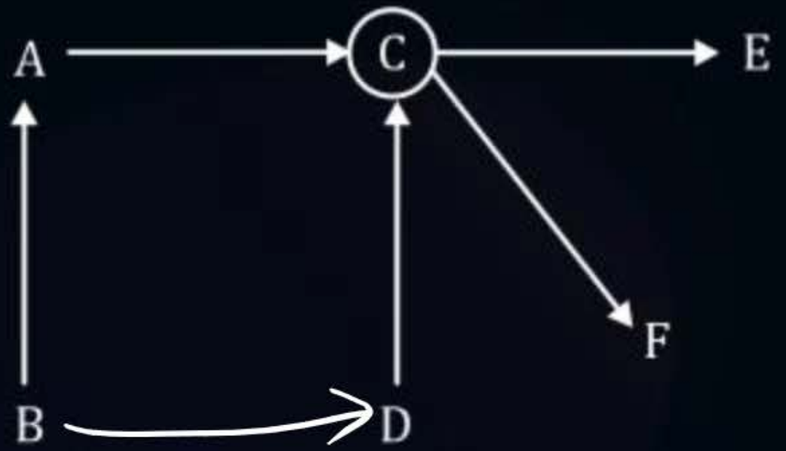
1. Apply DFS staring at any vertex (V): DFS (V)
2. Arrange all the vertices /node obtained in step DFS travels in the descending/decreasing order of their finishing times to get a valid topological sort ordering.



Topic : Graph Algorithms

Walkthrough/ Dry Run of Prev. Algo.

DAG:



Finishing times

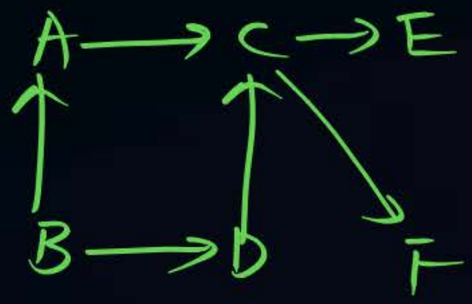
[redacted]

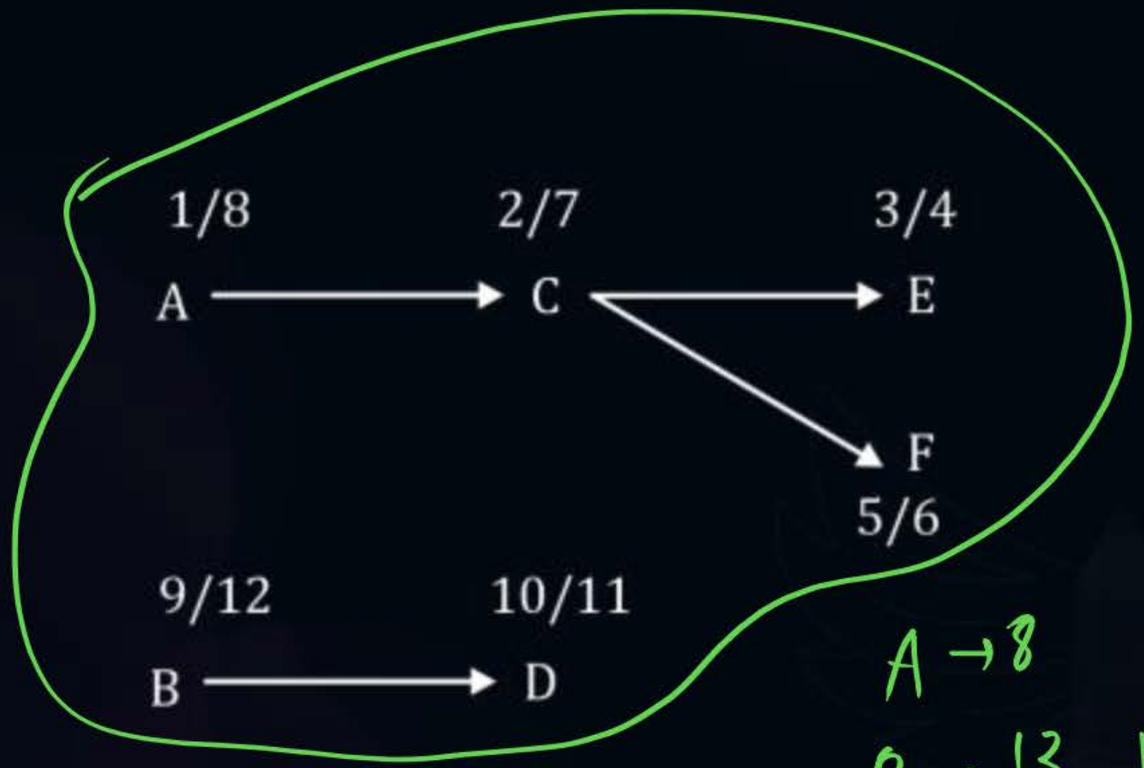


Topic : Graph Algorithms

Case-1:

DFS starting at A.





A → 8 D → 11

B → 12 E → 4

C → 7 F → 6

→ BDACFE

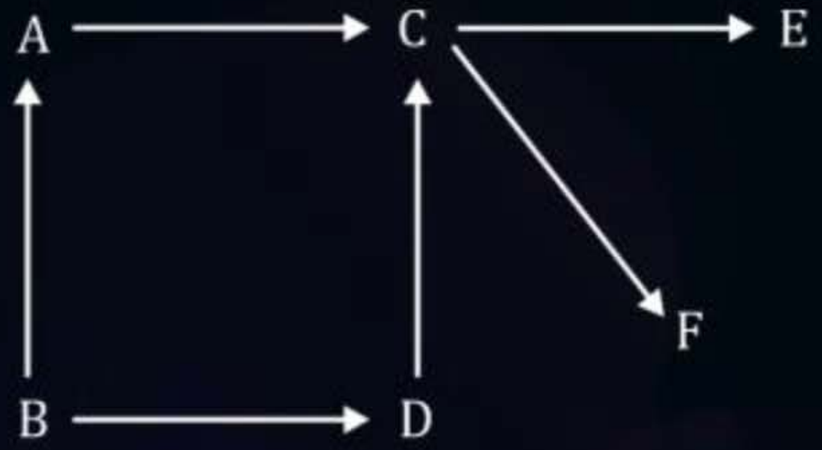


Topic : Graph Algorithms

Case-2:

DFS Starting at E

DAG



Finishing times

A→ 9	D → 11
B→ 12	E → 2
C→ 6	F → 5

1/2 3/6 4/5

E → C → F → DFS Spanning forest

7/12 8/9

B → A

D

10/11

Decreasing order: of Finishing time

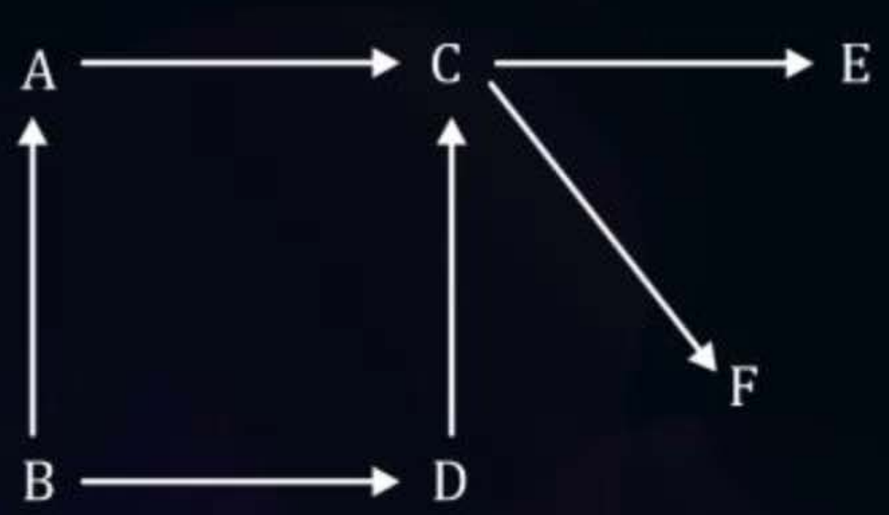
BDACFE



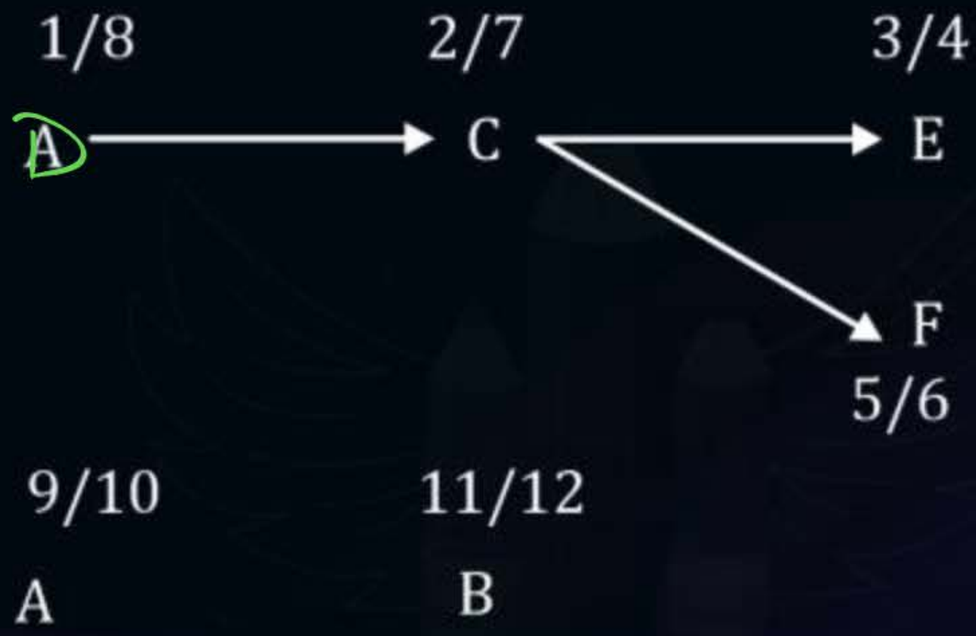
Topic : Graph Algorithms

Case-3:

Starting at D:



BADCFE

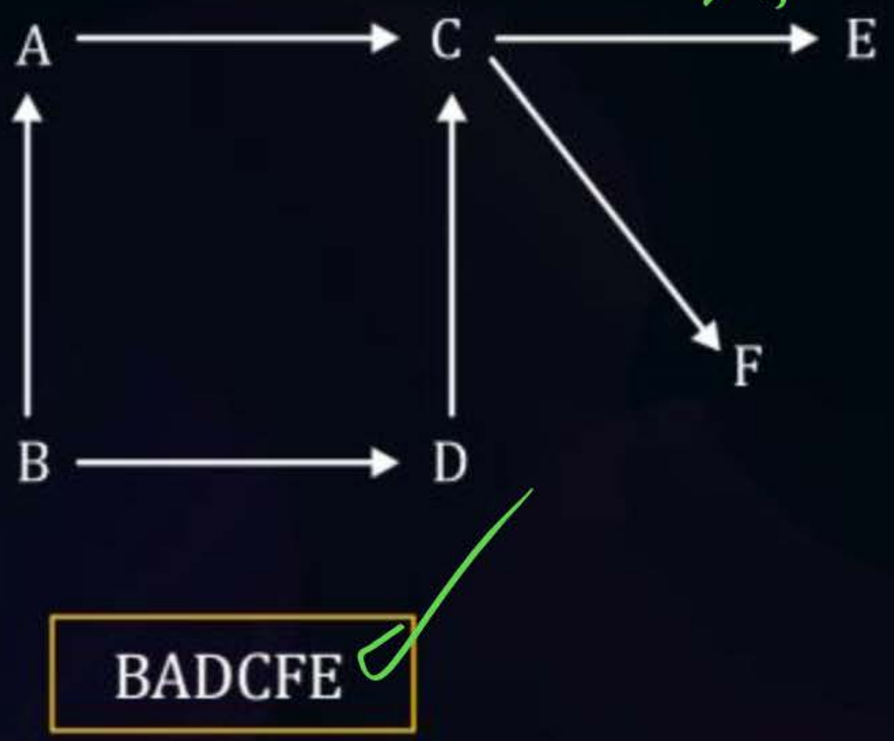




Topic : Graph Algorithms

Case-3:

Starting at D:



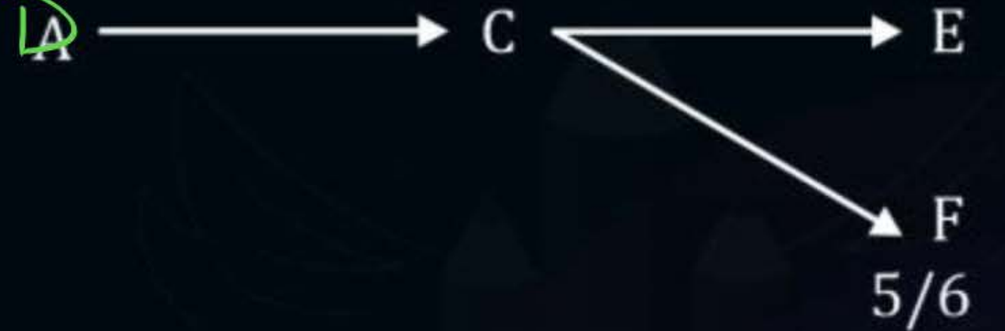
BADCFE

B 1/12 → D 2/9 → C 3/8 → E 4/5

C 3/8 → F 6/7

BADCFE

A 10/11



9/10 A

11/12 B



Topic : Graph Algorithms

#Q. Consider the depth-first-search of an undirected graph with 3vertices P, Q, & R. Let discovery time d(V) represent the time instant when the vertex 'V' is visited first, and finish time f(V) represent finishing time ; given

d(p) = 5 f(p) = 12
d(Q = 6 f(Q) = 10
d(R) = 14 f(R) = 18
Which is true?

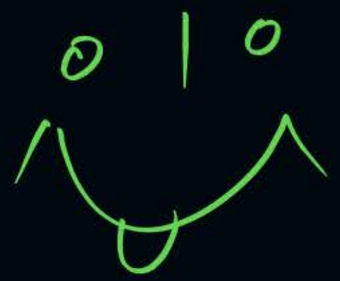
HW

A There is only one connected component.

B There are two connected components, P and R are connected

C There are two connected components, Q and R are

D There are two connected components, P and Q are connected



THANK - YOU